[강추] 파이썬 파일읽기 파일쓰기
https://blockdmask.tistory.com/454

가. 파일 생성하기

다음 코드를 IDLE 에디터로 작성하여 실행해 보자.

```
f = open("새파일.txt", 'w')


f.close()
```

프로그램을 실행한 디렉터리에 새로운 파일이 하나 생성된 것을 확인할 수 있을 것이다. 파일을 생성하기 위해 파이썬 내장 함수 open 을 사용했다. open 함수는 다음과 같이 "파일 이름"과 "파일 열기 모드"를 입력값으로 받고 결괏값으로 파일 객체를 리턴한다.

파일 객체 = open(파일 이름, 파일 열기 모드)

파일 열기 모드에는 다음과 같은 것들이 있다.

r	읽기모드 - 파일을 읽기만 할 때 사용
w	쓰기모드 - 파일에 내용을 쓸 때 사용
a	추가모드 - 파일의 마지막에 새로운 내용을 추가 시킬 때

파일을 쓰기 모드로 열면 해당 파일이 이미 존재할 경우 원래 있던 내용이 모두 사라지고, 해당 파일이 존재하지 않으면 새로운 파일이 생성된다. 위 예에서는 디렉터리에 파일이 없는 상태에서 새파일.txt 를 쓰기 모드인 'w'로 열었기 때문에 새파일.txt 라는 이름의 새로운 파일이 현재 디렉터리에 생성되었다.

만약 새파일.txt 파일을 `C:/doit` 디렉터리에 생성하고 싶다면 다음과 같이 작성해야 한다.

```
f = open("C:/doit/새파일.txt", 'w')

f.close()
```

위 예에서 `f.close()`는 열려 있는 파일 객체를 닫아 주는 역할을 한다. 사실 이 문장은 생략해도 된다. 프로그램을 종료할 때 파이썬 프로그램이 열려 있는 파일의 객체를 자동으로 닫아주기 때문이다. 하지만 `close()`를 사용해서 열려 있는 파일을 직접 닫아 주는 것이 좋다. 쓰기모드로 열었던 파일을 닫지 않고 다시 사용하려고 하면 오류가 발생하기 때문이다.

점프 투 파이썬**파일 경로와 슬래시(/)**

파이썬 코드에서 파일 경로를 표시할 때 `"C:/doit/새파일.txt"` 처럼 슬래시(/)를 사용할 수 있다. 만약 역슬래시(\)를 사용한다면 `"C:\\doit\\새파일.txt"` 처럼 역슬래시를 2 개 사용하거나 `r"C:\doit\새파일.txt"`와 같이 문자열 앞에 `r` 문자(Raw String)를 덧붙여 사용해야 한다. 왜냐하면 `"C:\note\test.txt"`처럼 파일 경로에 `\n`과 같은 이스케이프 문자가 있을 경우 줄바꿈 문자로 해석되어 의도했던 파일 경로와 달라지기 때문이다.

나. 파일을 쓰기 모드로 열어 내용 쓰기

위 예에서는 파일을 쓰기 모드로 열기만 했지 정작 아무것도 쓰지는 않았다. 이번에는 문자열 데이터를 파일에 직접 써 보자.

```
# writedata.py

f = open("C:/doit/새파일.txt", 'w')

for i in range(1, 11):

    data = "%d 번째 줄입니다.\n" % i

    f.write(data)

f.close()
```

위 프로그램을 다음 프로그램과 비교해 보자.

```
for i in range(1, 11):

    data = "%d 번째 줄입니다.\n" % i

    print(data)
```

두 프로그램의 다른 점은 data 를 출력하는 방법이다. 두 번째 방법은 우리가 계속 사용해 왔던 모니터 화면에 데이터를 출력하는 방법이고, 첫 번째 방법은 모니터 화면 대신 파일에 데이터를 적는 방법이다. 두 방법의 차이점은 print 함수 대신 파일 객체 f 의 write 함수를 사용한 것 말고는 없으니 바로 눈에 들어올 것이다.

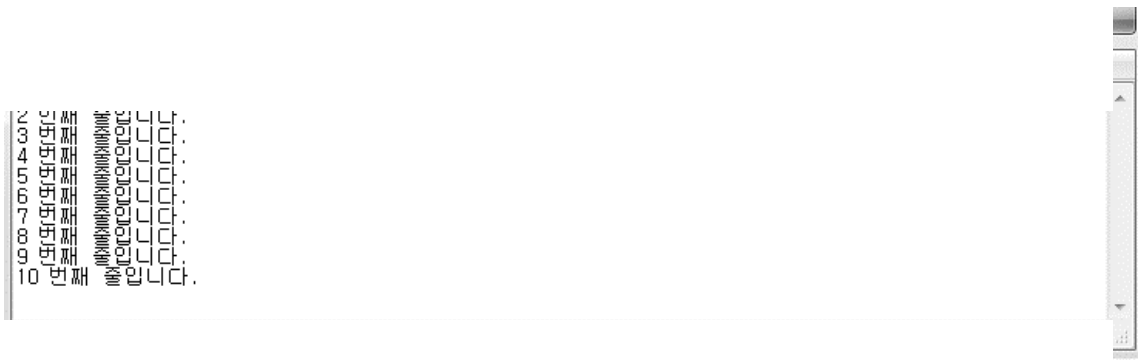
이제 명령 프롬프트 창에서 첫 번째 예제를 실행해 보자.

```
C:\Users> cd C:\doit

C:\doit> python writedata.py

C:\doit>
```

이 프로그램을 실행한 `C:/doit` 디렉터리를 살펴보면 새파일.txt 파일이 생성되었음을 확인할수 있다. 새파일.txt 파일에는 어떤 내용이 담겨 있는지 확인해 보자.



모니터 화면에 출력될 내용이 고스란히 파일에 들어 있는 것을 볼 수 있다.

다. 파일을 읽는 여러 가지 방법

파이썬에는 파일을 읽는 방법이 여러가지 있다. 이번에는 그 방법을 자세히 알아보자.

1) readline 함수 이용하기

첫 번째 방법은 readline 함수를 사용하는 방법이다. 다음 예를 보자.

```
# readline_test.py

f = open("C:/doit/새파일.txt", 'r')

line = f.readline()

print(line)

f.close()
```

위 예는 `f.open("새파일.txt", 'r')`로 파일을 읽기 모드로 연 후 `readline()`을 사용해서 파일의 첫 번째 줄을 읽어 출력하는 예제이다. 앞에서 만든 새파일.txt 를 수정하거나 지우지 않았다면 위 프로그램을 실행했을 때 새파일.txt 의 가장 첫 번째 줄이 화면에 출력될 것이다.

```
1 번째 줄입니다.
```

만약 모든 줄을 읽어서 화면에 출력하고 싶다면 다음과 같이 작성하면 된다.

```
# readline_all.py

f = open("C:/doit/새파일.txt", 'r')

while True:

    line = f.readline()
```

```
    if not line: break

    print(line)

f.close()
```

즉 while True: 무한 루프 안에서 f.readline()을 사용해 파일을 계속해서 한 줄씩 읽어 들인다. 만약 더 이상 읽을 줄이 없으면 break 를 수행한다(readline()은 더 이상 읽을 줄이 없을 경우 빈 문자열('')을 리턴한다).

한 줄 씩 읽어 출력할 때 줄 끝에 \n 문자가 있으므로 빈 줄도 같이 출력된다.

앞의 프로그램을 다음 프로그램과 비교해 보자.

```
while True:

    data = input()

    if not data: break

    print(data)
```

위 예는 사용자의 입력을 받아서 그 내용을 출력하는 경우이다. 파일을 읽어서 출력하는 예제와 비교해 보자. 입력을 받는 방식만 다르다는 것을 바로 알 수 있을 것이다. 두 번째 예는 키보드를 사용한 입력 방법이고, 첫 번째 예는 파일을 사용한 입력 방법이다.

2) readlines 함수 사용하기

두 번째 방법은 readlines 함수를 사용하는 방법이다. 다음 예를 보자.

```
f = open("C:/doit/새파일.txt", 'r')

lines = f.readlines()
```

```
for line in lines:

    print(line)

f.close()
```

readlines 함수는 파일의 모든 줄을 읽어서 각각의 줄을 요소로 갖는 리스트를 리턴한다. 따라서 위 예에서 lines 는 리스트 `["1 번째 줄입니다.\n", "2 번째 줄입니다.\n", ..., "10 번째 줄입니다.\n"]`가 된다. `f.readlines()`에서 `f.readline()`과는 달리 s 가 하나 더 붙어 있음에 유의하자.

점프 투 파이썬**줄 바꿈(\n) 문자 제거하기**

파일을 읽을 때 줄 끝의 줄 바꿈(\n) 문자를 제거하고 사용해야 할 경우가 많다. 다음처럼 strip 함수를 사용하면 줄 바꿈 문자를 제거할 수 있다.

```
f = open("C:/doit/새파일.txt", 'r')

lines = f.readlines()

for line in lines:

    line = line.strip()  # 줄 끝의 줄 바꿈 문자를 제거한다.

    print(line)

f.close()
```

3) read 함수 사용하기

세 번째 방법은 read 함수를 사용하는 방법이다. 다음 예를 보자.

```
f = open("C:/doit/새파일.txt", 'r')
```

```
data = f.read()

print(data)

f.close()
```

f.read()는 파일의 내용 전체를 문자열로 리턴한다. 따라서 위 예의 data 는 파일의 전체 내용이다.

4) 파일 객체를 for 문과 함께 사용하기

네 번째 방법은 파일 객체를 for 문과 함께 사용하는 방법이다.

```
f = open("C:/doit/새파일.txt", 'r')

for line in f:

    print(line)

f.close()
```

파일 객체(f)는 기본적으로 위와 같이 for 문과 함께 사용하여 파일을 줄 단위로 읽을 수 있다.

라. 파일에 새로운 내용 추가하기

쓰기 모드('w')로 파일을 열 때 이미 존재하는 파일을 열면 그 파일의 내용이 모두 사라지게 된다. 하지만 원래 있던 값을 유지하면서 단지 새로운 값만 추가해야 할 경우도 있다. 이런 경우에는 파일을 추가 모드('a')로 열면 된다. IDLE 에디터로 다음 소스 코드를 작성해 보자.

```
# adddata.py
```

```
f = open("C:/doit/새파일.txt",'a')

for i in range(11, 20):

    data = "%d 번째 줄입니다.\n" % i

    f.write(data)

f.close()
```

위 예는 새파일.txt 파일을 추가 모드('a')로 열고 write 를 사용해서 결괏값을 기존 파일에 추가해 적는 예이다. 여기에서 추가 모드로 파일을 열었기 때문에 새파일.txt 파일이 원래 가지고 있던 내용 바로 다음부터 결괏값을 적기 시작한다.

다음과 같이 작성한 코드를 실행해 보자.

```
C:\Users> cd C:\doit

C:\doit> python adddata.py

C:\doit>
```

새파일.txt 파일을 확인해 보면 원래 있던 내용 뒤에 새로운 내용이 추가되었음을 확인할 수 있다.

마. with 문과 함께 사용하기

지금까지 살펴본 예제를 보면 항상 다음과 같은 방식으로 파일을 열고 닫아 왔다.

```
f = open("foo.txt", 'w')

f.write("Life is too short, you need python")

f.close()
```

파일을 열면(open) 항상 닫아(close) 주어야 한다. 하지만 이렇게 파일을 열고 닫는 것을 자동으로 처리할 수 있다면 편리하지 않을까? 파이썬의 with 문이 바로 이런 역할을 해준다. 다음 예는 with 문을 사용해서 위 예제를 다시 작성한 모습이다.

```
with open("foo.txt", "w") as f:

    f.write("Life is too short, you need python")
```

위와 같이 with 문을 사용하면 with 블록(with 문에 속해있는 문장)을 벗어나는 순간 열린 파일 객체 f 가 자동으로 close 된다.

마지막 편집일시 : 2022 년 11 월 25 일 9:25 오전

1. 파이썬 파일 생성 open, close 함수

```
# 파일 열기
f = open('C:/Test/t1.txt', 'w')

# 파일에 텍스트 쓰기
f.write('blockdmask blog')
f.write('₩npython open func')
```

```python

# 파일 닫기
f.close()

```

Colored by Color Scripter

```python
# 파일 w 모드로 열기 (파일 새로 만듦)
f = open('C:/Test/t2.txt', 'w')

# write 함수를 이용해서 파일에 텍스트 쓰기
f.write('write write write₩n')
f.write('파이썬 파일 입출력 포스팅₩n')

# 파일 닫기
f.close()

```

Colored by Color Scripter cs

```python
# 파일 a 모드로 열기 (이미 있는 파일에 이어서 쓰기)
f = open('C:/Test/t2.txt', 'a')

# writelines 함수를 이용해서 파일에 문자열들 쓰기
f.writelines(['a', 'b', '123', '456', 'abcdefg', '₩n'])

# writelines 함수에 join 을 이용해서 문자열들에 자동 개행 넣기
f.writelines('₩n'.join(['BlockDMask', 'python', 'blog']))

# 파일 닫기
f.close()
```

Colored by Color Scripter cs

파이썬에서 파일을 생성할때에 사용하는 함수는 open() 함수 입니다.

open 함수는 파이썬 기본 내장함수 입니다.

```python
# 파일 r 모드로 열기
f = open('C:/Test/t2.txt', 'r')

# read() 함수 이용해서 하나씩 읽어오기
print('₩n1. read()')
print(f'위치 : {f.tell()}')

s1 = f.read(1)
print(s1)

# readline() 함수 이용해서 한 라인씩 읽어오기
print('₩n2. readline()')
print(f'위치 : {f.tell()}')

s2 = f.readline()
print(s2)

# readlines() 함수 이용해서 모두 읽어오기
print('₩n3. readlines()')
print(f'위치 : {f.tell()}')

s3 = f.readlines()
print(s3)

# 맨 처음 위치로 가서 한줄 읽기
print('₩n4. seek(0), readline()')

f.seek(0)
print(f'위치 : {f.tell()}')

s4 = f.readline()
print(s4)


# 파일 닫기
f.close()

```

Colored by Color Scripter cs

이 함수이름은 open 이지만 옵션에 따라서 파일을 생성 하고 열기인 경우, 읽기만을 위한 열기, 쓰기위한 열기 등 여러 옵션들이 있습니다.

1-1) open() 함수 기본 모양

def open(file, mode='r', buffering=None, encoding=None, errors=None, newline=None, closefd=True):

정의를 보면 이렇게 복잡하게 되어있지만 우리는 앞에 두가지만 알아보려 합니다. 마지 open 함수의 모양이 원래 아래와 같다고 생각해도 무방합니다. 뒤에있는 인자들은 저희같은 초보는 아직 몰라도 될것 같습니다.

def open(file, mode='r'):

open 함수는 파일을 열고 해당 파일에 대한 스트림을 반환합니다.

파일 열기를 실패했을때는 Error 를 발생시킵니다.

인자를 보면

첫번째 인자 file

- 파일 경로를 집어 넣습니다.

두번째 인자 mode

- 파일이 열리는 옵션, 모드 입니다. 이 모드에 따라 읽기 용인지 파일을 생성하고 쓸것인지 등이 정해집니다.

- 'r' : 기본값으로 정해져 있으며 파일을 읽기 위한 옵션 입니다.

- 'w' : 쓰기모드이며 파일에 내용을 쓸 때 사용하는 옵션 입니다. 만약 이미 파일이 존재하면 커서를 맨 앞으로 돌리면서 뒤에 내용을 다 잘라내기 때문에 내용이 사라질 수 있습니다. 파일이 존재하지 않는다면 새롭게 파일을 생성합니다.

- 'a' : 쓰기모드이며 파일에 내용을 쓸 때 사용하는 옵션입니다. w 옵션과는 달리 이미 파일이 존재하면 그 파일의 끝에 커서가 존재하고, 그 뒤에 이어쓰기가 가능합니다.

즉, 파일 내용을 잘라내지 않고 이어서 쓸 수 있습니다.

- 'x' : 파일이 없으면 파일을 생성하고 쓰기모드로 열립니다. 만약 파일이 있으면 에러를 발생시킵니다.

- 'b' : 바이너리 모드 입니다.

- 't' : 텍스트 모드 입니다. (기본값)

두번째 인자는 'r', 'w', 'a', 'x' 와 같이 't'나 'b'를 따로 쓰지 않으면 기본적으로 텍스트 모드인 't'로 열리게 됩니다. 즉, 읽기위해 'r'을 넣는다는것은 'rt'를 넣는것과 동일한 뜻 입니다.

만약 파일을 읽기모드인데 바이너리 모드로 열고싶다면 'rb' 이런식으로 집어 넣으면 됩니다.

따로 표기 하지않으면 기본적으로 텍스트 모드인 't'로 열립니다.

오픈한 파일을 닫으려면 파일 닫기 함수인 close()함수를 이용해야합니다.

1-2) open() 함수 예제

```
C: > Test > t1.txt
1   blockdmask blog
2   python open func
```

open('경로/파일이름', '모드') 함수를 이용해서 **'w'모드**로 열게되면 파일이 존재하지 않기 때문에 **해당 경로에 't1.txt'**라는 **파일이 생성된 후에 변수 f 에 파일 스트림이** 들어오게 됩니다.

해당 파일 스트림을 이용해서 우리는 파일에 내용을 쓰고, 읽고 할 수 있습니다.

예제에서는 **write(' ') 함수**를 통해서 파일에 문자열을 집어 넣어주었습니다.

파일을 뜯고 씹고 맛본 다음에는 파일을 얌전히 닫아줍니다. 이때 사용하는 함수는 **close() 함수** 입니다.

결과를 보면 t1.txt 라는 파일이 생성되고 그 안에 문자열이 잘 들어가있는걸 볼 수 있습니다.

**주의

파일 객체는 반드시 열었으면(open) 닫아야(close)합니다.

오픈한 파일을 닫지 않는다면 데이터가 날라갈 수 있습니다.

2. 파이썬 파일 쓰기 write, writelines 함수

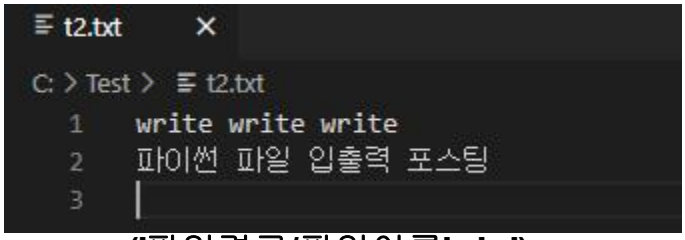
파일을 열었으면 파일 스트림을 이용해서 파일에 텍스트를 쓸 수 있습니다.

그 때 사용하는 함수가 write 함수, writelines 함수 입니다.

2-1) 파이썬 write 함수

IO.write("문자열")

write 함수는 매개변수로 파일에 넣을 문자열을 받습니다.



f = open('파일경로/파일이름', 'w')

- 해당 코드를 실행하면 t2.txt 파일이 생성됩니다. w 모드로 열었기 때문에 파일이 있으면 덮어씌워지고 내용은 싹 지워집니다. 파일이 없으면 파일이 생성됩니다.

f.write('')

- 파일을 열고 파일스트림을 이용해서 write 함수를 호출 했습니다.

내용을 보시면 개행문자(\n)을 직접 입력해준걸 볼 수 있습니다.
이렇게 개행이나 탭 등은 우리가 직접 입력을 해주어야합니다.

f.close()

- 파일을 열었으면 꼭 닫아야죠

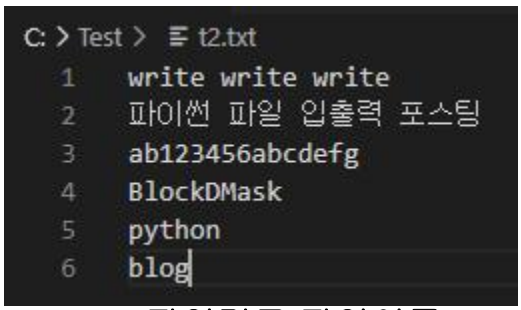
결과를 보면 파일에 우리가 입력한 내용이 잘 들어가있는걸 볼 수 있습니다.

2-2) 파이썬 writelines 함수

IO.writelines(List["문자열 1", "문자열 2", ...])

write 함수는 매개변수로 파일에 넣을 문자열 리스트를 받습니다.

아까 생성했던 t2.txt 에 이어서 파일에 문자열을 집어 넣어보겠습니다. 이미 있는 파일이고 해당 파일에 이어서 써야하니 'a'모드로 열어야겠죠?



f = open('파일경로/파일이름', 'a')

- 'a' 모드로 파일을 열었기 때문에 파일이 있으면 해당 파일의 맨 끝으로 가서 이어서 쓸 수 있습니다. 파일이 없으면 파일이 생성됩니다.

f.writelines(list[문자열 1, 문자열 2, ...])

- 파일을 열고 파일스트림을 이용해서 writelines 함수를 호출 했습니다.

내용을 보시면 이번에도 문자열 리스트의 맨 끝에 개행문자(\n)을 직접 입력해준걸 볼 수 있습니다.

그럼 우리는 join 함수를 이용해서 개행을 문자열 리스트 사이사이에 넣어주는 걸 응용할 수 있습니다. '\n'.join([문자열 리스트]) 이런식으로 하면 위 결과창과 같이 사이사이 개행이 잘 들어가 있는걸 볼 수 있습니다.

f.close()

- 파일을 열었으면 꼭꼭 닫아야죠 22

3. 파이썬 파일 읽기 read, readline, readlines, seek, tell 함수

파일을 읽는 함수 **read, readline, readlines** 세가지와 파일 위치 관련함수 **seek, tell** 함수 이렇게 한번 알아보겠습니다.

3-1) read, readline, readlines 함수 설명

IO.read(n)

read(n) 함수는 파일 스트림으로 부터 해당 위치의 문자 n 개를 읽어오는 함수 입니다. n 바이트를 읽어오는 것 입니다. read() 이렇게 아무것도 넣지 않으면 모든 문자를 읽어오게 됩니다.

반환 : 문자 N 개를 반환합니다.

IO.readline()

readline() 함수는 함수 이름에서 알 수 있듯이 파일 스트림으로부터 해당 위치에서부터 한 줄, 한 라인의 문자열을 읽어옵니다. 즉 '\n' 개행이 있을때 까지 문자열을 읽어옵니다.

반환 : 문자열을 반환합니다.

IO.readlines()

readlines() 함수는 해당 위치에서부터 파일의 모든 문자열을 읽어옵니다. (개행 포함)

반환 : List[문자열]을 반환합니다. 리스트로 반환됩니다.

3-2) seek, tell 함수 설명

우리가 워드나 메모장 하다못해 어딘가에 글을 쓸때 항상 세로줄로 | 깜빡깜빡 하는 커서가 있을것 입니다. 이것은 현재 글을 쓸 위치를 알려주는것인데, 이것을 기준으로 글이 써지게 됩니다.

파이썬에서 파일을 다룰때에도 이러한 커서의 위치에 따라서 그 위치에 문자를 쓰고, 그 위치에서 문자를 읽어오게 됩니다. **seek, tell 함수는 이 커서의 위치를 이동하고, 커서의 현재 위치**를 가지고 오는 함수 입니다.

IO.seek(위치)

seek 함수는 해당 위치로 파일의 커서를 옮깁니다. 파일의 맨 처음 위치는 0 입니다.

IO.tell()

tell 함수는 현재 커서의 위치를 반환하는 함수입니다.

3-3) 파일 읽기 함수 예제

우선 txt 파일을 준비해줍니다. 파일 쓰기 예제에서 사용했던 t2.txt 을 읽어오겠습니다.

현재 t2.txt 파일은 이러한 상태입니다.

```
C: > Test > t2.txt
1  write write write
2  파이썬 파일 입출력 포스팅
3  ab123456abcdefg
4  BlockDMask
5  python
6  blog
```

```
1. read()
위치 : 0
w

2. readline()
위치 : 1
rite write write


3. readlines()
위치 : 19
['파이썬 파일 입출력 포스팅\n', 'ab123456abcdefg\n', 'BlockDMask\n', 'python\n', 'blog']

4. seek(0), readline()
위치 : 0
write write write
```

1. read()를 보면 파일 커서(=파일포인터)의 위치가 0 에서 read()를 한번 호출하니 파일로 부터 "w"를 읽어와서 반환 하는 것을 볼 수있습니다.

2. readline()을 보면 앞서 사용했던 read() 함수에 의해서 파일 커서의 위치가 1 로 변경된걸 알 수 있습니다. 그 상태에서 readline() 함수를 이용하니

"write write write" 문자열에서 1 개 이동한 위치에서 읽어오기 때문에 "rite write write" 를 읽어온걸 알 수 있습니다.

3. realines() 를 보면 앞서 사용했던 realine 때문에 파일 커서의 위치가 바뀐걸 볼 수 있습니다. 그 상태에서 파일의 끝까지 문자들을 읽어오고, 개행까지 포함해서 리스트로 변경해주는걸 볼 수 있습니다.

4. seek(0) 함수를 이용해서 파일 커서의 위치를 맨 앞으로 옮기고 readline() 함수를 이용해서 한 라인의 문자열을 읽어 들였습니다. "write write write" 문자열을 읽어온것을 확인 할 수 있습니다.

이상으로 파이썬에서 파일을 생성하고, 파일 내용을 읽고 쓰는 함수들에 대해서 알아보았습니다. 감사합니다.

==========
==========

Python 파일 및 디렉토리 삭제하는 방법 Python 에서 파일을 삭제하는 방법으로 os 모듈의 remove() 함수가 많이 사용됩니다. 간단한 사용방법은 아래와 같습니다.

os.remove(path, option)

첫번째 인자값으로 파일위치 및 이름을 입력하고, 두번째 인자는 선택옵션으로 디렉토리와 관련된 값을 설정하는데 사용됩니다. 참고로 path 설정에는 상대값이 사용될 수 있습니다. (Ex – ./img/test.txt)

! 파일 삭제 예제보기, os.remove()

만약 아래의 경로에 다음과 같은 파일이 있고 이를 삭제해보도록 하겠습니다. 경로값은 상대경로이며 ./abc/에 test.txt 이름의 파일이 존재합니다.@ 경로 – /abc/test.text

파일 삭제를 위해서 아래의 커맨드를 사용합니다.

```
os.remove('/abc/test.text')
```

이처럼 간단하게 파일을 삭제할 수 있습니다. 파일 삭제의 명령어를 사용하는 경우 아래의 프로세스를 생각해봐야합니다.

- 1. 파일의 유무를 우선적으로 확인
- 2. 파일이 있는 경우 파일을 삭제
- 3. 삭제 결과를 출력

위 과정에서는 파일의 유무를 확인하였는데 파일 경로에 해당 파일이 없는 경우 에러가 발생하기 때문입니다. 이런 이유를 파일 삭제 명령어를 수행하기 이전에 파일 유무를 확인하는 것이 좋은 방법입니다. 그렇다면 위 프로세스를 따르는 간단한 예제를 하나 더 알아보죠.

! Python 파일 유무 확인 후 파일 삭제하기

아래의 코드는 os.path.isfile()을 사용하여 파일의 유무를 먼저 확인합니다. 파일이 존재하는 경우 True 를 반환하고 이때에만 삭제하는 명령어를 추가하였습니다.

```
file = './upload/test.txt'


ifos.path.isfile(file):
 os.remove(file)


 return'okay'
```

이제 파일이 존재하는 경우에만 파일 삭제가 이루어지므로 삭제 에러를 방지할 수 있습니다.

! 디렉토리 삭제하는 방법, os.rmdir()

디렉토리를 삭제하는 경우 os.rmdir()이 사용됩니다.

os.rmdir(path, option)

바.

사용방법은 위의 예제와 비슷합니다. 아래의 예제를 보세요. 만약 디렉토리 /abc/를 삭제하려면? 아래와 같이 사용합니다.

```
os.rmdir('/abc/')
```

이제 디렉토리를 삭제하는 방법도 알아봤습니다.